

AI Shopping Assistant

Setup guide — from an empty machine to a running application. Every command is written out; nothing is assumed to be installed already.

What you are installing	Detail
Backend	Spring Boot 3.5.15 monolith on Java 21, serving a REST API on port 8080
Frontend	React 19 + Vite + TypeScript, dev server on port 5173
Database	H2, created automatically as a file in the project. No install needed
AI model	Runs locally through Ollama. No API keys, no cloud, no cost
Total time	About 15 minutes, most of it downloading the language model

Read this first. The application needs **two terminals** running at the same time — one for the backend, one for the frontend. Leave both open while you use the app. Closing a terminal stops that half of the application.

Before you start: what must be installed

Four things. Install each one, then run its check command and confirm you get a version number rather than an error.

Software	Version needed	Check it with	Where to get it
Git	Any recent	<code>git --version</code>	git-scm.com/downloads
Java JDK	21 or newer	<code>java -version</code>	adoptium.net (choose Temurin 21)
Node.js	20.19+ or 22.12+	<code>node -v</code>	nodejs.org (choose LTS)
Ollama	Any recent	<code>ollama --version</code>	ollama.com/download

You do not need to install these

Not needed	Why
Maven	The project includes the Maven wrapper. The mvnw script downloads the correct Maven version on first use.
MySQL or any database	H2 runs inside the application and writes to a file. A MySQL profile exists but is optional.
An OpenAI or Anthropic API key	The language model runs on your own machine through Ollama.

If a check command says "not recognized" or "command not found" right after you installed something, close every terminal window and open a new one. A terminal reads the system PATH when it starts, so one opened before the install cannot see the new program.

STEP 1 Download the two AI models

Ollama installs a background service that starts on its own. These two commands download the models the application uses. This is the slow part — roughly 5 GB in total — so start it first and let it run while you read on.

```
# The chat model - about 4.7 GB
ollama pull qwen2.5:7b

# The embedding model, used for search - about 274 MB
ollama pull nomic-embed-text
```

Confirm both arrived:

```
ollama list
```

You should see qwen2.5:7b and nomic-embed-text listed. If the command hangs or is refused, start the Ollama service manually with `ollama serve` and try again.

STEP 2 Clone the repository

Pick any folder you like, then:

```
git clone https://github.com/AdityaAnand0108/ai-shopping-assistant.git
cd ai-shopping-assistant
```

Every command from here on is run **from inside this folder**, unless a step says otherwise.

STEP 3 Start the backend (terminal 1)

Use the command for your operating system. Do not install Maven — the mvnw script in the project handles it.

Windows (PowerShell or Command Prompt)

```
.\mvnw.cmd spring-boot:run
```

macOS or Linux

```
./mvnw spring-boot:run
```

The first run takes 2–4 minutes. It downloads Maven and every Java dependency, then creates the database, loads 60 products, and builds the search index. Later runs take about 10 seconds.

Wait until you see these lines. The last one is the signal it is ready:

```
Migrating schema "public" to version "1 - core schema"
...
Seeded 60 products
Seeded 4 demo accounts: [satvik, sarah, rahul, demo]
Seeded 9 demo orders
Indexed 60 products in 6942ms
```

Do not skip the wait. The web server starts about a second before the demo data finishes loading. If you open the app in that gap, the catalog looks empty. Wait for Indexed 60 products.

STEP 4 Start the frontend (terminal 2)

Open a **second** terminal. Leave the first one running — the backend must stay up. Navigate to the same project folder, then:

```
# From inside the ai-shopping-assistant folder
cd frontend

# Downloads the frontend packages. First time only, about 30 seconds
npm install

# Starts the dev server
npm run dev
```

You should see:

```
VITE v8.2.2 ready in 335 ms

-> Local: http://localhost:5173/
```

STEP 5 Open the application

In a browser, go to:

```
http://localhost:5173
```

The catalog appears immediately and needs no account — browsing is public, as in any real shop. To use the assistant or view orders, sign in with one of the demo accounts below.

Username	Password	What this account shows
satvik	Password123	5 orders covering every status - the fullest account to explore
sarah	Password123	3 orders, one out for delivery
rahul	Password123	1 cancelled order
demo	Demo1234	No orders - shows the empty state

These are synthetic accounts in a local demo database. The passwords are stored hashed; they are printed here only so the application can be run.

STEP 6 Check it works

Sign in as satvik, open **Assistant**, and ask:

```
What Nike t-shirts do you have?
```

The first reply takes 10–20 seconds because the model is loading into memory; later replies take 1–10 seconds. A correct answer lists real products with real prices, and shows a line beneath it reading Based on: searched the catalog. That line is the point of the project: it tells you the answer came from a real database call rather than from the model guessing.

Other things you can do

Run the test suite

All 159 tests run without Ollama and without a database server — they use an in-memory database and a stub model. This is the quickest way to confirm a clone is healthy, and it works even if the model download has not finished.

```
# Windows
.\mvnw.cmd clean verify

# macOS / Linux
./mvnw clean verify
```

Start over with fresh data

The database and search index live in a data folder inside the project. Stop the backend, delete that folder, and start it again — everything is recreated from scratch.

```
# Windows
rmdir /s /q data

# macOS / Linux
rm -rf data
```

Use a real MySQL database instead of H2

Optional. Requires a running MySQL 8 server. The same migrations work unchanged.

```
./mvnw spring-boot:run -Dspring-boot.run.profiles=mysql
```

Override DB_HOST, DB_PORT, DB_NAME, DB_USER and DB_PASSWORD as environment variables if your server is not on localhost with the default credentials.

Useful URLs while the application is running

URL	What it is
http://localhost:5173	The application
http://localhost:8080/swagger-ui.html	Interactive API documentation

URL	What it is
<code>http://localhost:8080/actuator/health</code>	Health check, including whether the AI model is reachable
<code>http://localhost:8080/api/products</code>	The catalog as raw JSON, no sign-in required

Troubleshooting

These are the problems people actually hit, in the order they tend to hit them.

Symptom	Cause and fix
"java is not recognized" , or the same for node, git or ollama	The terminal was opened before you installed it. Close all terminal windows and open a new one. If it still fails, the installer did not add the program to your PATH — reinstall and accept the default options.
"Port 8080 was already in use"	Another copy of the backend is still running. On Windows, open Task Manager and end the <code>java.exe</code> processes, or run: <code>taskkill /F /IM java.exe</code> . On macOS or Linux: <code>lsof -ti:8080 xargs kill -9</code> .
The catalog page is empty	You opened the app before the backend finished loading its demo data. Wait for <code>Indexed 60 products</code> in terminal 1, then refresh.
"The assistant is unavailable" when you send a message	Ollama is not running, or the chat model was not downloaded. Run <code>ollama list</code> and confirm <code>qwen2.5:7b</code> is there. If the command itself fails, run <code>ollama serve</code> .
The first reply takes 20 seconds	Expected. The model is being loaded into memory on first use. Subsequent replies are much faster. If every reply is slow, the machine is short of RAM — 16 GB is comfortable, 8 GB will struggle.
"Could not reach the server" in the browser	The backend is not running. Check terminal 1 for errors and restart it.
npm install fails	Check <code>node -v</code> reports 20.19 or higher. Older versions cannot build this frontend. If it still fails, delete the <code>frontend/node_modules</code> folder and try again.
Signing in says "Invalid username or password" for a demo account	Passwords are case sensitive: <code>Password123</code> with a capital P. Note that five wrong attempts lock an account for 15 minutes.
The assistant answers, but a reply carries an amber warning	Working as designed. The application checks every answer against what the database actually returned, and flags any figure it cannot verify. It is telling you the model said something unsupported.

Stopping the application

Press **Ctrl + C** in each terminal. On Windows the backend sometimes leaves a child process behind that keeps port 8080 open; if the next start complains about the port, end the leftover `java.exe` as described above.

Nothing in this project reaches the internet after setup. The model runs locally, the database is a file inside the project folder, and no API keys are required at any point.